

9

Implementing End-to-End Tests Using Playwright

In the previous chapters, we have written unit tests for our backend using Jest. Now, we are going to learn how to write and run end-to-end tests on our user interface using Playwright. First, we set up Playwright in our project and VS Code to allow for running frontend tests. Then, we are going to write some frontend tests for our application. Next, we are going to learn about reusing test setups with fixtures. Finally, we are going to learn how to view test reports and run Playwright in CI using GitHub Actions.

In this chapter, we are going to cover the following main topics:

- Setting up Playwright for end-to-end testing
- Writing and running end-to-end tests
- Reusable test setups using fixtures
- Viewing test reports and running in CI

Technical requirements

Before we start, please install all requirements from [***Chapter 1**, Preparing for Full-Stack Development*](#), and [***Chapter 2**, Getting to Know Node.js and MongoDB*](#).

The versions listed in those chapters are the ones used throughout the book. While installing a newer version should not be an issue, please note that certain steps might work differently on a newer version. If you are having an issue with the code and steps provided in this book, please try

using the versions mentioned in the *Technical requirements* section of *Chapters 1* and *2*.

You can find the code for this chapter on GitHub:

<https://github.com/PacktPublishing/Modern-Full-Stack-React-Projects/tree/main/ch9>.

If you cloned the full repository for the book, Husky may not find the `.git` directory when running `npm install`. In that case, just run `git init` in the root of the corresponding chapter folder.

The CiA link for this chapter can be found at:

<https://youtu.be/WjwEwUR8g2c>

Setting up Playwright for end-to-end testing

Playwright is a test runner to facilitate end-to-end testing on various web rendering engines, such as Chromium (Chrome, Edge, Opera, etc.), WebKit (Safari), and Firefox. It can run tests on Windows, Linux, and macOS, locally or on CI. There are two ways of running Playwright:

- **Headed:** Opens a browser window where it can be seen what Playwright is doing
- **Headless:** Runs the rendering engine in the background and only displays the results of the tests in the Terminal or a generated test report

In this chapter, we are going to explore both ways of running Playwright. Let's now install Playwright in our project.

Installing Playwright

To install Playwright, we can use `npm init playwright`, which runs a command that installs Playwright, creates a folder for end-to-end tests for us, adds a GitHub Actions workflow to run tests in CI, and installs Playwright browsers so it can run tests in various engines. Follow these steps to install Playwright:

1. Copy the existing **ch8** folder to a new **ch9** folder, as follows:

```
$ cp -R ch8 ch9
```

2. Open the **ch9** folder in VS Code and open a new Terminal.
3. Run the following command:

```
$ npm init playwright@1.17.131
```

NOTE

*Usually, it is a good idea to install the latest version here by running **npm init playwright@latest**. However, to make sure the instructions in this book are reproducible even when new versions with breaking changes are released, we pin the version here.*

4. When asked if you want to proceed with installing the **create-playwright** package, press *Return/Enter* to confirm it. Then select **JavaScript**. As for the directory name, keep the **tests** default name and press *Return/Enter* to confirm it. Type **y** to add a GitHub Actions workflow. Type **y** again to install Playwright browsers. It will now take a while to download and install the different browser engines.
5. We need to adjust some files to make Playwright work with ES modules. Edit **playwright.config.js** and change the line with the **require()** import at the beginning of the file to the following:

```
import { defineConfig, devices } from '@playwright/test'
```

6. Also, change the export from **module.exports** to the following:

```
export default defineConfig({
```

7. Delete the **tests-examples/** folder and the **tests/example.spec.js** file.

After installing Playwright, we need to prepare our backend for end-to-end testing, so let's do that now.

Preparing the backend for end-to-end testing

To prepare the backend for end-to-end testing, we need to start an instance of the backend with the in-memory MongoDB server, similarly to what we did for the Jest tests. Let's do that now:

1. Create a new **backend/src/e2e.js** file. Inside it, import **dotenv**, **globalSetup**, and the **app** and **initDatabase** functions:

```
import dotenv from 'dotenv'
dotenv.config()
import globalSetup from './test/globalSetup.js'
import { app } from './app.js'
import { initDatabase } from './db/init.js'
```

2. Then, define a new **async** function to run a testing server:

```
async function runTestingServer() {
```

3. Inside this function, we first run the **globalSetup** function, which runs an in-memory MongoDB server. Then, we initialize the database and run the Express app:

```
  await globalSetup()
  await initDatabase()
  const PORT = process.env.PORT
  app.listen(PORT)
  console.info(`TESTING express server running on http://localhost:${PORT}`)
}
```

4. Finally, we run the defined function:

```
runTestingServer()
```

5. Edit **backend/package.json** and add a new script to run the **e2e.js** file:

```
"e2e": "node src/e2e.js",
```

6. In the root of the project, install **concurrently**, a tool to run two commands in parallel:

```
$ npm install --save-dev concurrently@8.2.2
```

We are going to use this tool to run the backend and frontend in parallel.

7. Edit **package.json** in the root of the project and define an **e2e** script that will run **e2e:client** and **e2e:server** scripts in parallel:

```
"e2e": "concurrently \"npm run e2e:client\" \"npm run e2e:server\"",
```

8. Now, define the **e2e:client** script, in which we just run the prebuilt frontend:

```
"e2e:client": "npm run build && npm run start",
```

For performance reasons, we do not run the dev server. Otherwise, we would be slowing down our end-to-end tests. We could omit the build script here, but then we must remember to build our frontend after making changes before running tests, and we must do this in CI as well. Alternatively, when running the tests locally, especially when we are only running certain tests and not all of them, we could run the dev server instead of building.

9. Then, we define the **e2e:server** script, which runs the **e2e** script in the **backend** folder:

```
"e2e:server": "cd backend/ && npm run e2e",
```

10. Edit **playwright.config.js** and set the **baseURL** by changing the following line:

```
use: {  
  /* Base URL to use in actions like `await page.goto('/')`. */  
  baseURL: 'http://localhost:5173',  
}
```

11. Finally, edit **playwright.config.js** and *replace* the **webServer** config at the bottom of the file with the following:

```
webServer: {
```

```
command: 'npm run e2e',  
url: 'http://localhost:5173',  
},
```

Now that we have successfully set up Playwright and prepared the back-end for end-to-end testing, let's get started writing and running end-to-end tests!

Writing and running end-to-end tests

We are now going to write and run our first end-to-end test with Playwright. Let's start with a simple test, which just verifies that we have properly optimized our title for search engines. Follow these steps to write and run your first end-to-end test:

1. Create a new **tests/seo.spec.js** file. Inside this file, we are going to check whether the title of our page is set properly.
2. Inside this newly created file, first import the **test** and **expect** functions from **@playwright/test**:

```
import { test, expect } from '@playwright/test'
```

3. Then, we define a test in which we check whether the title of the blog is set properly:

```
test('has title', async ({ page }) => {
```

As you can see, the **test** function is similar to how we defined tests in Jest. Playwright additionally allows us to access special contexts in our test, called **fixtures**. The **page** fixture is the most essential fixture in Playwright and allows us to access browser features and interact with a page.

4. Inside the test, we first navigate to the URL of our frontend by using the **page.goto** function:

```
await page.goto('/')
```

5. Then, we use the **expect** function to check whether the page displays the correct title:

```
await expect(page).toHaveTitle('Full-Stack React Blog')
})
```

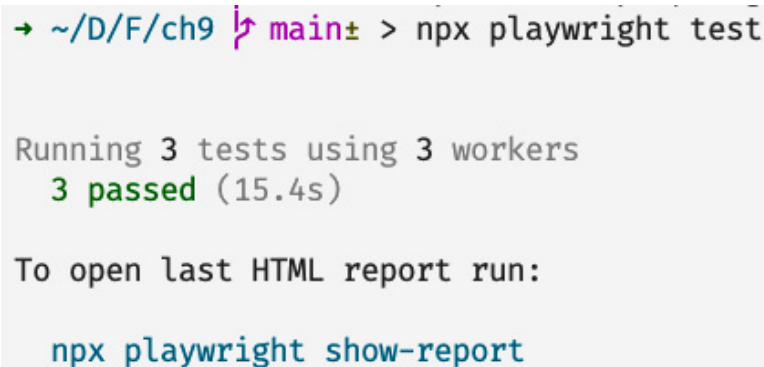
As we can see, the syntax of Playwright is very similar to Jest. We also have an **expect** function to make assertions, such as the page having a certain title.

6. Before running the tests, make sure the **dbserver** Docker container is running.
7. We can now run this test by opening a new Terminal and executing the following command:

```
$ npx playwright test
```

Make sure you are in the root of our project (**ch9** folder), and not inside the **backend** folder, when running this command!

You will see that Playwright runs our test three times (on Chromium, Firefox, and Webkit) and that all of them passed successfully. The following screenshot shows the result of running Playwright in the command line:

A terminal window showing the execution of a Playwright test. The prompt is '~ /D/F/ch9 main >'. The command entered is 'npx playwright test'. The output shows 'Running 3 tests using 3 workers', '3 passed (15.4s)', and a message to open the HTML report with the command 'npx playwright show-report'.

```
→ ~/D/F/ch9 main > npx playwright test

Running 3 tests using 3 workers
  3 passed (15.4s)

To open last HTML report run:
  npx playwright show-report
```

Figure 9.1 – Running our first test in Playwright!

Now that we have successfully executed our test, let's move on to running tests using the VS Code extension.

Using the VS Code extension

Instead of manually running all tests via the command line, we can also run specific tests (or all tests) using a VS Code extension, similar to what we did for Jest. Additionally, the extension also allows us to get a visual overview of which tests are succeeding (or not), allows us to inspect tests while running in a browser, and can even record our interactions in the browser and generate tests from it!

Let's start by setting up the VS Code extension and running our test from it:

1. Open the **Extensions** tab in VS Code and search for **Playwright**.
2. Click the **Install** button to install **Playwright Test for VS Code** by Microsoft.
3. Click on the **Testing** tab in VS Code (the flask icon), which we also used for the Jest extension. Here, you will now see **Jest** and **Playwright** in the list.
4. Expand the **Playwright | tests** path, click on **seo.spec.js** to load the file, and then click on the **Play** icon next to **seo.spec.js** to run the test.

As we can see in the following screenshot, the test was executed successfully, and all tests are passing:

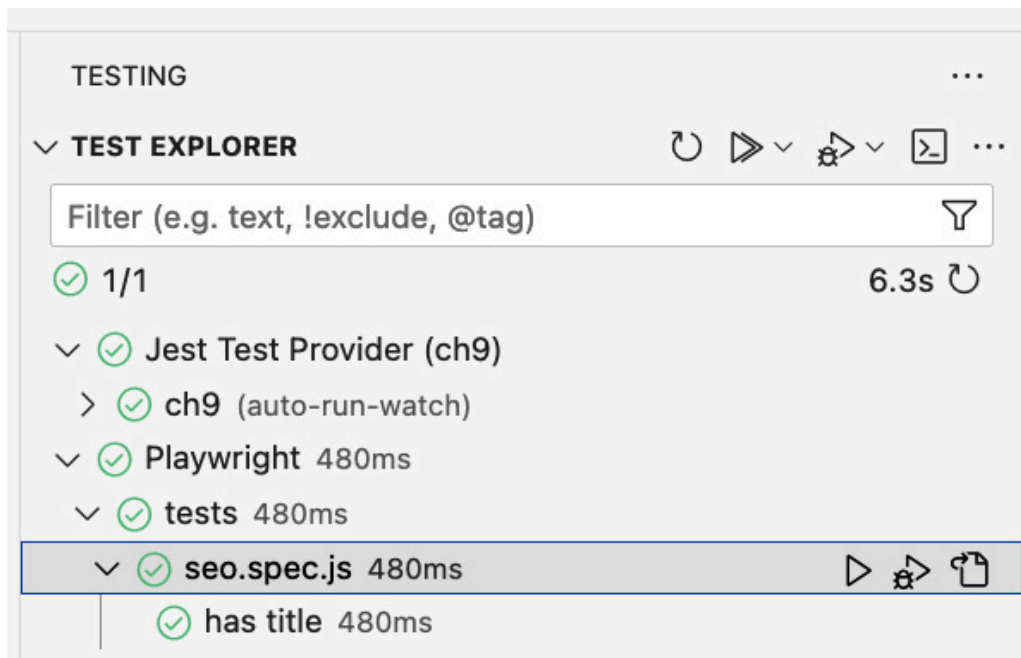


Figure 9.2 – Our playwright test successfully running from the VS Code extension!

Now that we have successfully run the tests in the VS Code extension in headless mode, let's move on to running them in headed mode, where we show what Playwright is doing in the browser while running tests.

Showing the browser while running tests

The Playwright VS Code extension also has a useful **Show browser** option, which opens the browser while running the tests. This allows us to debug tests or the frontend while the tests are running. Let's try it out now:

1. On the bottom of the **Testing** sidebar, check the **Show browser** box at the bottom of the sidebar and run the test again.

A browser window will open and run the test. However, our test is very quick and simple, so it runs within a short amount of time and there is not much to see.

2. To better inspect the test, we can use the trace viewer. Check **Show trace viewer** at the bottom of the **Testing** sidebar and run the test again. You will see the following window open:

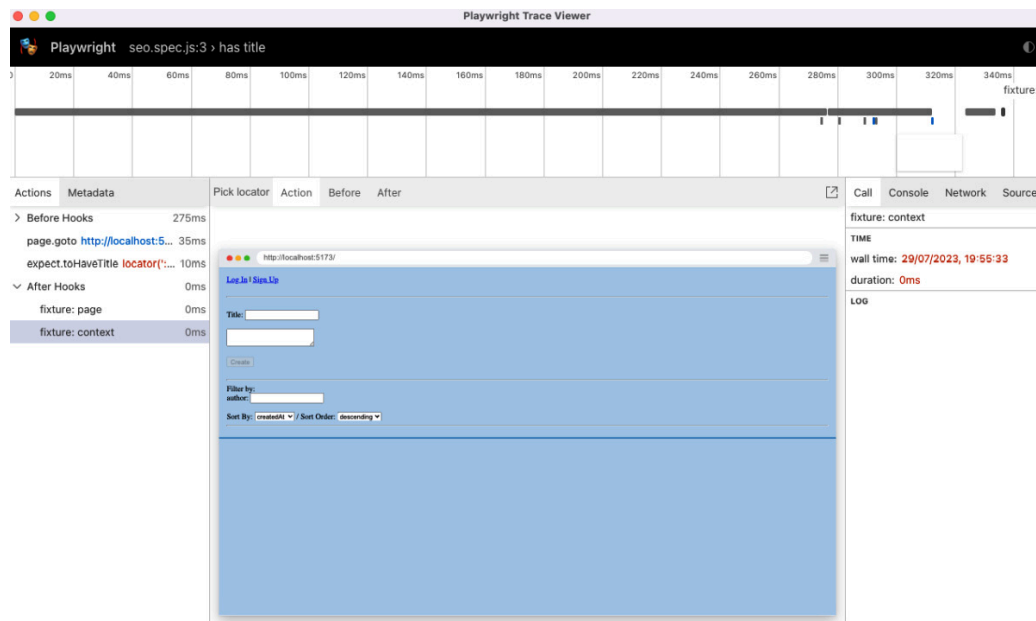


Figure 9.3 – The Playwright trace viewer

As we can see, the Playwright trace viewer shows us that the test ran **page.goto** and then **expect.toHaveTitle**. It also shows the state of the app at each step of the test. In our case, we only have one step though.

This feature is especially useful when developing larger and more complex tests.

NOTE

*It is also possible to run Playwright in UI mode, which opens the Playwright app in a separate window that allows us to run the tests separately and look at them being executed, similar to using the **Show trace viewer** function in the VS Code extension. You can run Playwright in UI mode by executing the following command: `npm playwright test --ui`*

Now that we have learned about using the extension to run tests, we can move on to a very useful feature of the extension: recording actions to create a new test. Let's do that now.

Recording a test

The Playwright extension can also record new tests. Let's now create a new test for the signup page by using the test recording functionality of the VS Code extension:

1. Unlike running a Playwright test, the test recorder does not automatically start our frontend and backend, so we need to start them manually first. Open a new Terminal and execute the following command:

```
$ npm run e2e
```

2. In the bottom section of the **Testing** sidebar, click on **Record new**. A browser window should open.
3. In the browser window, navigate to the frontend by pasting `http://localhost:5173/` into the URL bar.

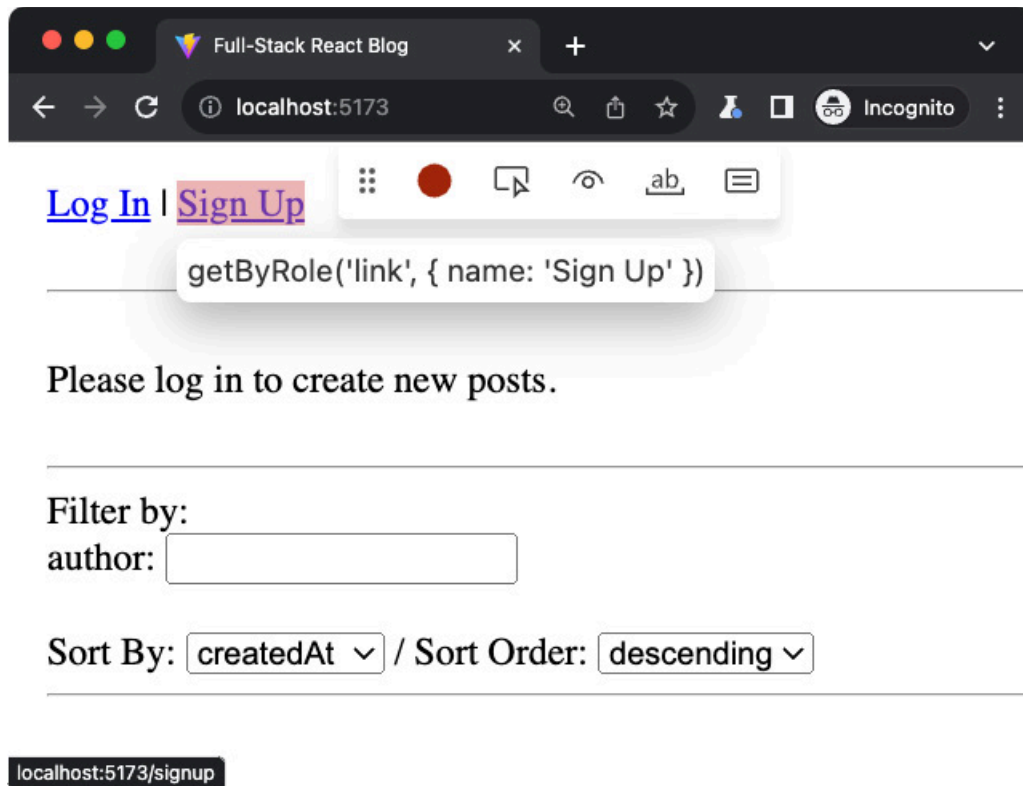


Figure 9.4 – The Playwright test recorder while hovering over the “Sign Up” link

4. Then, click on the **Sign Up** link. The sign-up page should open.
5. On this new page, enter a username and password; for example, **test** and **test**. Then press the **Sign Up** button.
6. You will get redirected to the login page. Now, log in with the same username and password as before.
7. You will get redirected to the main page and will be logged in as **test**. You can now close the browser window. You will see that inside VS Code you now have a new **test-1.spec.ts** file containing all the actions we just did in the browser!
8. Save the file and quit the **e2e** script running the backend and frontend. You will now see the **test-1.spec.ts** file in the **Testing** sidebar. If you try running the test, you will notice that it gets stuck on the login part, because our test currently does not wait for the redirect to the login page.

While recording tests is a useful feature to speed up writing end-to-end tests, it cannot always write functioning tests for us. We now need to clean up our recorded test and add assertions to it.

For reference, here is the full code generated by the Playwright test recorder:

```
import { test, expect } from '@playwright/test'
test('test', async ({ page }) => {
  await page.goto('http://localhost:5173/')
  await page.getByRole('link', { name: 'Sign Up' }).click()
  await page.getByLabel('Username:').click()
  await page.getByLabel('Username:').fill('test')
  await page.getByLabel('Password:').click()
  await page.getByLabel('Password:').fill('test')
  await page.getByRole('button', { name: 'Sign Up' }).click()
  await page.getByLabel('Username:').click()
  await page.getByLabel('Username:').fill('test')
  await page.getByLabel('Password:').click()
  await page.getByLabel('Password:').fill('test')
  await page.getByRole('button', { name: 'Log In' }).click()
})
```

Now that we have recorded a test, let's clean it up to make it run properly.

Cleaning up and finalizing the recorded test

If you have a look at the test, you will see that it has all the actions we performed in the browser, but it does not verify that we have been successfully logged in. It also does not wait for pages to finish loading and some matchers do not match the proper text. Let's fix those problems now:

1. Rename **tests/test-1.spec.ts** to **tests/auth.spec.js**.
2. Edit **tests/auth.spec.js** and rename the test to **allows sign up and log in**:

```
test('allows sign up and log in', async ({ page }) => {
```

3. We need to define a unique username to be able to run our test multiple times without having to restart the backend to clear the MongoDB memory server:

```
const testUser = 'test' + Date.now()
```

NOTE

It is important to not sign up twice with the same username, as the in-memory MongoDB database is reused for all the tests. Make sure that tests can run independently from each other and do not rely on data from other test files, as the test files could run in any order. Only the order within a single test file is guaranteed. Using `Date.now()` returns the current time in milliseconds and is mostly collision-safe as long as we do not run too many tests in parallel. For a more collision-safe solution, you could use a UUID generator instead.

4. Change the `page.goto()` URL to `/` to ensure it uses the `baseURL` we set up earlier:

```
await page.goto('/')
```

5. Fill in the generated username when signing up:

```
await page.getByLabel('Username:').fill(testUser)
```

6. After the click on the **Sign Up** button, wait for the URL to update by using the following function:

```
await page.getByRole('button', { name: 'Sign Up' }).click()
await page.waitForURL('**/login')
```

Waiting for the next page to load is necessary because the recording does not support page-loading detection right now and it would otherwise fire the commands on the old page or during the redirect, which would cause the test to fail.

7. For the login, we also fill in the generated username:

```
await page.getByLabel('Username:').fill(testUser)
```

8. After that, make the test click on on the **Log In** button and wait for the URL to update again:

```
await page.getByRole('button', { name: 'Log In' }).click()
await page.waitForURL('**/')
```

9. To more easily match the **Header** React component, edit

src/components/Header.jsx and turn the `<div>` elements into `<nav>` elements:

```
export function Header() {
  const [token, setToken] = useAuth()
  if (token) {
    const { sub } = jwtDecode(token)
    return (
      <nav>
        Logged in as <User id={sub} />
        <br />
        <button onClick={() => setToken(null)}>Logout</button>
      </nav>
    )
  }
  return (
    <nav>
      <Link to='/login'>Log In</Link> | <Link to='/signup'>Sign Up</Link>
    </nav>
  )
}
```

10. At the end of the test, we now add an assertion that checks whether the Header (`<nav>` element) contains the text **Logged in as** and the generated username:

```
await expect(page.locator('nav')).toContainText('Logged in as ' + testUser)
})
```

Using **toContainText** instead of **toContainText** ensures that the text does not have to be exactly the provided string. In our case, the **Logout** text is also part of the `<nav>` element, so the full text would be **Logged in as testXXXXLogout**.

11. Run the test either using the VS Code extension or by running the **npm playwright test** command in the Terminal (whichever you prefer), and you will see that it passes successfully now!

NOTE

If the test does not execute successfully, you might have accidentally recorded some additional actions and not cleaned them up properly. Compare your test to the code example provided by this book to make sure the test is properly defined and cleaned.

Now that we know how defining basic tests works in Playwright, let's learn about reusable test setups using fixtures.

Reusable test setups using fixtures

After creating the authentication test, you might be thinking: what if I want to define a test for creating a new post now? We would have to first sign up, then log in, then create the post. This is quite tedious and the more complex our tests get; the more tedious defining tests would get. Fortunately, Playwright has a solution for these kinds of problems. Playwright introduces a concept called fixtures, which are contexts for the test that can contain reusable functions. For example, we could define an **auth** fixture to provide sign-up and log-in functions to all tests.

When we used Jest, we were using before/after hooks to prepare the common environment for multiple tests. Fixtures have some advantages over before/after hooks. Mainly, they encapsulate setup and teardown in the same place and are reusable between test files, composable, and more flexible. Additionally, fixtures are provided *on demand*, which means that Playwright will only set up the fixtures necessary for running a certain test.

Playwright also includes some fixtures out of the box, which we are going to learn about now.

Overview of built-in fixtures

Playwright comes with some built-in fixtures, one of which we have already learned about: the **page** fixture. We are now going to briefly introduce the most important built-in fixtures Playwright provides out of the box:

- **browser**: Allows controlling browser features, such as opening a new page
- **browserName**: Contains the name of the browser that is currently running the test
- **page**: By far the most important built-in fixture, used to control interactions with the page, visiting URLs, matching elements, doing actions, and much more
- **context**: An isolated context for the current test run
- **request**: Used to make API requests from Playwright

Now that we have learned about the built-in fixtures Playwright provides, let's continue by defining our own fixture.

Writing our own fixture

Signing up and logging in are common actions that we will need to do often in our end-to-end tests, so they are the perfect case for creating a fixture. Follow these steps to create a new **auth** fixture:

1. Create a new **tests/fixtures/** folder.
2. Inside it, create a new **tests/fixtures/AuthFixture.js** file, where we define an **AuthFixture** class:

```
export class AuthFixture {
```

3. This class will receive the **page** fixture in the constructor:

```
  constructor(page) {  
    this.page = page  
  }
```

4. Define a **signUpAndLogIn** method, which follows the actions from the auth test to generate a unique username, then sign up and log in the user:

```
  async signUpAndLogIn() {  
    const testUser = 'test' + Date.now()  
    await this.page.goto('/signup')  
    await this.page.getByLabel('Username:').fill(testUser)  
    await this.page.getByLabel('Password:').fill('password')
```



```

    await this.page.getByRole('button', { name: 'Sign Up' }).click()
    await this.page.waitForURL('**/login')
    await this.page.getByLabel('Username:').fill(testUser)
    await this.page.getByLabel('Password:').fill('password')
    await this.page.getByRole('button', { name: 'Log In' }).click()
    await this.page.waitForURL('**/')
    return testUser
  }
}

```

5. Create a new **tests/fixtures/index.js** file. Inside it, import the **test** function from Playwright (renaming it to **baseTest**) and the **AuthFixture** we just defined:

```

import { test as baseTest } from '@playwright/test'
import { AuthFixture } from './AuthFixture.js'

```

6. Then, define and export a new **test** function, extending the **baseTest** function from Playwright by defining a new **auth** fixture inside it:

```

export const test = baseTest.extend({
  auth: async ({ page }, use) => {
    const authFixture = new AuthFixture(page)
    await use(authFixture)
  },
})

```

TIP

*It is also possible to do additional setup of the fixture context before calling the **use()** function, and additional breakdown after calling it. This can be used to, for example, create a set of example posts before executing tests and deleting them again afterward. If the backend had a way to delete a user, creating a temporary user and deleting the created username again after using the fixture would be a better option to deal with the issue of username collisions.*

7. Additionally, re-export the **expect** function from Playwright to make it easier to import from our fixtures:

```
export { expect } from '@playwright/test'
```

Now that we have defined our custom fixture, let's use it while creating a new test!

Using custom fixtures

We are now going to define an end-to-end test for creating a new post. To create a post, we need to be logged in so we can use our **auth** fixture for preparing the environment. Follow these steps to define the new test and use our custom fixture:

1. Create a new **tests/create-post.spec.js** file. To use the custom fixture, we now need to import the **test** and **expect** functions from the **fixtures/index.js** file:

```
import { test, expect } from './fixtures/index.js'
```

2. Define a new test to verify that post creation works, using the **page** and **auth** fixtures:

```
test('allows creating a new post', async ({ page, auth }) => {
```

3. We can now use the **signUpAndLogIn** method from our custom **auth** fixture to create and log in a new user:

```
    const testUser = await auth.signUpAndLogIn()  
  })
```

4. We can make use of Playwright code generation again to record our test. First, save the file and execute the **create-post.spec.js** test with **Show browser** enabled.
5. Then, create a new line after the **auth.signUpAndLogIn** function is called and press **Record at cursor**.
6. Now we can record actions from the already opened browser window (which is also already logged in, because the fixture methods were called already!). Click into the title field and enter **Test Post** as the post title, then press **Tab** to go to the next field, enter **Hello World!** as

the post content, then press *Tab* again and press *Return/Enter* to create a new post.

NOTE

The post does not actually get created, because Playwright closes the backend right after it finishes running it, so at the time of recording, the backend is already shut down. If you want to record with the backend running, explore the `webServer.reuseExistingServer` setting in `playwright.config.js`.

7. Go back to the file, and you will see that all actions were properly recorded! The following code should have been recorded:

```
await page.getByLabel('Title:').click()
await page.getByLabel('Title:').fill('Test Post')
await page.getByLabel('Title:').press('Tab')
await page.locator('textarea').fill('Hello World!')
await page.locator('textarea').press('Tab')
await page.getByRole('button', { name: 'Create' }).press('Enter')
```

8. Now, we just need to add a check whether the post was created successfully:

```
await expect(page.getByText(`Test PostWritten by ${testUser}`)).toBeVisible()
})
```

As we are controlling the test environment, it is enough to check that the text **Test PostWritten by testXXX** (without a space between “Post” and “Written”) is visible on the page. This will tell us that the post was created in the list.

9. Run the test, and you will see that it passes successfully!

We could create additional fixtures for handling posts (creating, editing, deleting) and use these to, for example, verify that the link to a single post works properly and adjusts the title accordingly. However, extending end-to-end tests like that is similar to what we have already done and is thus left as an exercise for you.

Viewing test reports and running in CI

After successfully creating some end-to-end tests for our blog application, let's wrap up the chapter by learning how to view HTML test reports and how to run Playwright in CI.

Viewing an HTML report

Playwright automatically generates HTML reports of test runs. We can generate these by executing the following command to run all tests:

```
$ npx playwright test
```

Then, run the following command to serve and view the HTML report of the last run:

```
$ npx playwright show-report
```

The report should open in a new browser window, and look as follows:

Q

All 9Passed 9Failed 0Flaky 0Skipped 0

Total time: 7.0s

▼ auth.spec.js

✓ allows sign up and log in chromium764ms
auth.spec.js:3

✓ allows sign up and log in firefox1.4s
auth.spec.js:3

✓ allows sign up and log in webkit880ms
auth.spec.js:3

▼ create-post.spec.js

✓ allows creating a new post chromium772ms
create-post.spec.js:3

✓ allows creating a new post firefox1.2s
create-post.spec.js:3

✓ allows creating a new post webkit653ms
create-post.spec.js:3

▼ seo.spec.js

✓ has title chromium303ms
seo.spec.js:3

✓ has title firefox722ms
seo.spec.js:3

✓ has title webkit324ms
seo.spec.js:3

Figure 9.5 – An HTML test report generated by Playwright

As we can see, our three tests were successfully run on all three browsers. Click on one of the test runs to view all executed test steps in detail.

Running Playwright tests in CI

When we initialized Playwright, we were asked if we want to generate a GitHub Actions CI file. We agreed, so Playwright automatically generated a CI configuration for us in the `.github/workflows/playwright.yml` file. This workflow checks out the repository, installs all dependencies, installs Playwright browsers, runs all Playwright tests, and then uploads the report as an artifact so it can be viewed from the CI run. We still need to adjust the CI workflow to also install dependencies for our backend, so let's do that now:

1. Edit `.github/workflows/playwright.yml` and add the following step to it:

```
- name: Install dependencies
  run: npm ci
- name: Install backend dependencies
  run: cd backend/ && npm ci
```

The `npm ci` command ensures that the project already has a `package-lock.json` file and does not write a lock file, ensuring a clean state for CI to run on.

2. Add, commit, and push everything to a GitHub repository to see Playwright running in CI.

NOTE

*Make sure to create a new repository from just the contents of the **ch9** folder (not the whole **Full-Stack-React-Projects** folder!), otherwise GitHub Actions will not detect the `.github` folder.*

3. Go to GitHub, click on the **Actions** tab, select the **Playwright Tests** workflow on the sidebar, and then click on the latest workflow run.
4. At the bottom of the run, there is an **Artifacts** section, which contains a `playwright-report` object that can be downloaded to view the HTML report.

The following screenshot shows the Playwright tests running in GitHub Actions, with the report provided as an artifact:

The screenshot displays the GitHub Actions interface for a workflow named 'ci: define playwright tests #4'. The workflow is in a 'Success' state, triggered by a push to the 'main' branch. The summary shows a total duration of 1m 58s and one artifact named 'playwright-report' with a size of 418 KB. The 'Run details' section shows a single job named 'test' that completed successfully in 1m 48s. The 'Artifacts' section lists the 'playwright-report' artifact.

Triggered via	Status	Total duration	Billable time	Artifacts
omnidan pushed -> 4a15ad2 main	Success	1m 58s	2m	1

Name	Size
playwright-report	418 KB

Figure 9.6 – Playwright running in GitHub Actions

As we can see, running Playwright in CI by using the provided template is simple and straightforward.

Summary

In this chapter, we learned about using Playwright for end-to-end testing. We first set up Playwright in our project and prepared our backend for end-to-end testing. Then, we wrote and ran our first test. Next, we learned about fixtures to make reusable test contexts. Finally, we viewed the generated HTML report and set up CI to run Playwright, generate a report, and save it as an artifact in the pipeline.

In the next chapter, [***Chapter 10**, Aggregating and Visualizing Statistics Using MongoDB and Victory*](#), we are going to learn how to aggregate data using MongoDB and expose this aggregated data via a backend. Then, we are going to consume the aggregated data in the frontend and visualize it using Victory with various visualization types.

